

A Single `mongod` Becomes Unavailable in a Shard

Replica sets (page 559) provide high availability for shards. If the unavailable `mongod` is a *primary*, then the replica set will *elect* (page 580) a new primary. If the unavailable `mongod` is a *secondary*, and it disconnects the primary and secondary will continue to hold all data. In a three member replica set, even if a single member of the set experiences catastrophic failure, two other members have full copies of the data.¹¹

Always investigate availability interruptions and failures. If a system is unrecoverable, replace it and create a new member of the replica set as soon as possible to replace the lost redundancy.

All Members of a Replica Set Become Unavailable

If all members of a replica set within a shard are unavailable, all data held in that shard is unavailable. However, the data on all other shards will remain available, and it's possible to read and write data to the other shards. However, your application must be able to deal with partial results, and you should investigate the cause of the interruption and attempt to recover the shard as soon as possible.

One or Two Config Servers Become Unavailable

Three distinct `mongod` instances provide the *config servers* (page 670).

If one or two config servers become unavailable, the cluster's metadata becomes *read only*. You can still read and write data from the shards, but no *chunk migration* (page 684) or *chunk splits* (page 723) will occur until all three servers are available. Replace the config server as soon as possible. If all config databases become unavailable, the cluster can become inoperable.

If the config servers are inconsistent, the balancer will not perform any *chunk migration* (page 684) nor will the `mongos` perform *auto-chunk splits* (page 723).

Note: All config servers must be running and available when you first initiate a *sharded cluster*.

Renaming Config Servers and Cluster Availability

If the name or address that a sharded cluster uses to connect to a config server changes, you must restart **every** `mongod` and `mongos` instance in the sharded cluster. Avoid downtime by using CNAMEs to identify config servers within the MongoDB deployment.

To avoid downtime when renaming config servers, use DNS names unrelated to physical or virtual hostnames to refer to your *config servers* (page 670).

Generally, refer to each config server using the DNS alias (e.g. a CNAME record). When specifying the config server connection string to `mongos`, use these names. These records make it possible to change the IP address or rename config servers without changing the connection string and without having to restart the entire cluster.

Shard Keys and Cluster Availability

The most important consideration when choosing a *shard key* are:

- to ensure that MongoDB will be able to distribute data evenly among shards, and
- to scale writes across the cluster, and

¹¹ If an unavailable secondary becomes available while it still has current oplog entries, it can catch up to the latest state of the set using the normal *replication process*, otherwise it must perform an *initial sync*.